ЛР3. Обчислювальний експеримент і трансферне навчання

Дедлайн: до початку лекції 11. **Оцінювання:** 10 балів. **Стек:** Python 3.12, PyTorch, torchvision, NumPy, scikit-learn, Matplotlib, Pillow. Навчання у `float32` на будь-якому доступному пристрої (CPU, CUDA, MPS); тестова оцінка - на CPU.

Мета роботи

Порівняти три режими навчання ResNet-18 на малій вибірці за однакового обчислювального бюджету й відповісти на питання: **яку частину виграшу дають попередньо навчені ознаки, яку - донавчання кістяка під нову задачу, і чи більша кожна з цих різниць за розкид між запусками?**

Стартового коду немає: завантаження даних, поділ, модель, режими, цикл навчання, перевірки й аналіз ви пишете самостійно. Тому дані, поділ, архітектура, режими та гіперпараметри нижче зафіксовано однозначно, а для поділу й режимів навчання наведено контрольні числа. Розбіжність із ними означає помилку реалізації, а не особливість вашого запуску: знайдіть її, перш ніж іти далі.

Предмет роботи - протокол експерименту: однакові умови для всіх режимів, кілька зерен на режим, одноразова тестова оцінка й висновок, що враховує шум. Негативний, але коректно виміряний і пояснений результат («різниця не відрізняється від розкиду») не знижує оцінку.

Вихідні дані

Набір Intel Image Classification (посилання). Після розпакування архіву використовуються дві теки:

```
seg_train/seg_train/<клас>/*.jpg   14 034 зображення
seg_test/seg_test/<клас>/*.jpg      3 000 зображень
```

Теку `seg_pred` не використовуйте. Порядок класів фіксований: `buildings`, `forest`, `glacier`, `mountain`, `sea`, `street` (індекси 0–5). Зображення відкривайте через Pillow з `.convert("RGB")`. Майже всі зображення мають розмір 150×150, але 55 із 17 034 - менші за висотою (наприклад, 150×113); їх не відкидайте, розмір вирівнюють перетворення.

Поділ

Поділ будується за наведеною процедурою: контрольна таблиця нижче відповідає саме їй. Інший детермінований поділ, зокрема через `train_test_split`, дасть інші файли.

1. Для кожного класу візьміть список імен файлів його теки, **відсортований як рядки** - так їх упорядковують `sorted()` і `ImageFolder`: `0.jpg`, `10006.jpg`, `1001.jpg`, ...
2. **train.** Створіть `rng = np.random.default_rng(0)` один раз і використовуйте його для всіх класів по черзі, у фіксованому порядку класів. Для класу k з n_k файлами в `seg_train`: `idx =`

`np.arange(n_k)`, `rng.shuffle(idx)`; у train ідуть файли з номерами `idx[:200]` у відсортованому списку кроку 1. Разом 1200 зображень.

3. **validation і test.** Створіть новий `np.random.default_rng(0)` і так само, одним генератором для всіх класів по черзі, переберіть класи `seg_test`: `idx = np.arange(n_k)`, `rng.shuffle(idx)`, `cut = n_k // 2`; файли з номерами `idx[:cut]` - validation, `idx[cut:]` - test.

Збережіть поділ у `split.csv` зі стовпцями `file` (ім'я файлу без шляху; імена в наборі унікальні), `label` (назва класу) і `part` (`train`, `validation` або `test`). Формат і порядок рядків - на ваш розсуд; далі всі дані завантажуються **лише** за цим файлом.

Контроль. Порівняйте свій поділ із таблицею: кількість зображень і сума числових імен файлів (`20057.jpg` → 20057) для кожного класу й частини. У кожній клітинці - «кількість / сума». Збіг усіх 18 клітинок практично гарантує правильний поділ; клітинка, що не збігається, показує, де шукати помилку.

Клас	train	validation	test
`buildings`	200 / 1 996 761	218 / 4 837 323	219 / 4 837 559
`forest`	200 / 2 032 625	237 / 5 276 962	237 / 5 291 326
`glacier`	200 / 2 183 923	276 / 6 135 652	277 / 6 157 250
`mountain`	200 / 1 950 345	262 / 5 789 535	263 / 5 849 765
`sea`	200 / 1 987 613	255 / 5 655 068	255 / 5 668 011
`street`	200 / 1 841 817	250 / 5 541 845	251 / 5 547 533
Разом	1200 / 11 993 084	1498 / 33 236 385	1502 / 33 351 444

Перетворення

Використовуйте `torchvision.transforms`; параметри, не названі нижче, - за замовчуванням.

Частина	Перетворення
train	`RandomResizedCrop(160, scale=(0.7, 1.0))` → `RandomHorizontalFlip()` → `ToTensor()` → `Normalize(mean, std)`
validation, test	`Resize((160, 160))` → `ToTensor()` → `Normalize(mean, std)`

$\text{mean} = (0{,}485;\ 0{,}456;\ 0{,}406)$, $\text{std} = (0{,}229;\ 0{,}224;\ 0{,}225)$ - статистики ImageNet, на яких попередньо навчено ваги. Ті самі числа використовуються і в режимі `scratch`, щоб режими відрізнялися лише ініціалізацією й тим, що навчається. Інших аугментацій і тестової аугментації немає.

Модель і режими

Кожен запуск будує модель у такому порядку:

1. `torch.manual_seed(seed)` і `np.random.seed(seed)`.

2. `model = torchvision.models.resnet18(weights=W)`, де `W = None` для `scratch` і `ResNet18_Weights.IMAGENET1K_V1` для `frozen` та `unfreeze`.
3. `model.fc = torch.nn.Linear(512, 6)` зі стандартною ініціалізацією.
4. `requires_grad=False` для всіх параметрів, крім перелічених у стовпці «Що навчається» таблиці режимів.
5. Перенесення на пристрій і створення оптимізатора.

Режим	Ваги	Що навчається (`requires_grad=True`)	Параметрів, що навчаються	Групи AdamW: параметри, швидкість навчання	У `eval()` під час навчання
`scratch`	випадкові	уся мережа	11 179 590	усі параметри, $3 \cdot 10^{-3}$	-
`frozen`	ImageNet	`fc`	3 078	`fc`, $3 \cdot 10^{-3}$	`conv1`, `bn1`, `layer1`–`layer4`
`unfreeze`	ImageNet	`layer4`, `fc`	8 396 806	`fc`, $3 \cdot 10^{-3}$; `layer4`, $3 \cdot 10^{-4}$	`conv1`, `bn1`, `layer1`–`layer3`

Загальна кількість параметрів моделі в усіх режимах - 11 179 590. Оптимізатор - `torch.optim.AdamW` з `weight_decay=1e-4` в усіх групах; решта аргументів - за замовчуванням. Параметри, що не навчаються, в оптимізатор не передаються.

Гіперпараметри пакетної нормалізації лишайте за замовчуванням (`momentum=0.1`, `eps=1e-5`). **Не використовуйте:** розкладів швидкості навчання, обрізання градієнта, змішаної точності, згладжування міток, зважування класів, ранньої зупинки, вибору епохи чи контрольної точки за validation.

Цикл навчання

- **Завантажувачі.** train: `batch_size=32`, `shuffle=True`, `drop_last=False`, `num_workers=0`, без власного `generator` чи `sampler`: перемішування бере випадковість із глобального генератора PyTorch, засіяного на кроці 1. validation і test: `batch_size=64`, `shuffle=False`, `num_workers=0`, порядок - як у вашому `split.csv`.
- **Бюджет.** 8 епох по $\lceil 1200/32 \rceil = 38$ кроків, разом **304 кроки** оптимізатора; останній пакет епохи містить 16 зображень.
- **Епоха.** На початку кожної епохи - `model.train()`, потім `.eval()` для кожного модуля зі стовпця «У `eval()`». Оформіть це окремою функцією; далі в тексті вона називається `set_train_mode`. На кожному пакеті: `zero_grad()`, прямий прохід, `CrossEntropyLoss()` (середнє за пакетом), `backward()`, `step()`. Втрата епохи - $\sum_b L_b \, |b| \, / \, 1200$, де L_b - втрата пакета b до кроку оптимізатора.
- **Після кожної епохи** - `model.eval()`, `torch.no_grad()`, логіти validation і macro-F1:

$$\mathrm{F1}_k = \frac{2\,TP_k}{2\,TP_k + FP_k + FN_k}, \qquad \text{macro-F1} = \frac{1}{6}\sum_{k=0}^{5} \mathrm{F1}_k,$$

де TP_k, FP_k, FN_k рахуються для класу k проти решти. Дозволено `sklearn.metrics.f1_score(average="macro")`.

Підсумок запуску $F_{r,s}$ - macro-F1 на validation **після 8-ї епохи** для режиму r і зерна s. Використовуються ваги після 8-ї епохи.

Повне навчання на GPU чи MPS займає кілька хвилин на запуск, на CPU - довше. Подбайте про час заздалегідь.

---

1. Перевірки до навчання

Поділ. Наведіть таблицю контролю поділу для вашого `split.csv` і позначте, чи збігаються всі числа. Переконайтеся, що жоден файл не входить у дві частини.

Пакет після перетворень. Збережіть рисунок із перших 16 зображень будь-якого навчального пакета в тому вигляді, в якому їх отримує модель: після аугментацій, з оберненим нормуванням і підписаною міткою над кожним.

Початкова втрата. Для режимів `scratch` і `frozen` побудуйте модель за кроками 1–5 зі зерном 1 і обчисліть втрату L_0 на першому пакеті навчального завантажувача до жодного кроку оптимізатора: після `set_train_mode`, під `torch.no_grad()` (прямий прохід у режимі навчання оновлює ковзні статистики BN, тому цей екземпляр моделі далі не використовується). Порівняйте з $\ln 6 \approx 1{,}792$ - втратою моделі, що дає всім класам однакову ймовірність. Поясніть відхилення, якщо воно є.

---

2. Перевірка режимів навчання

Перевірки цього пункту оцінюються в першій складовій рубрики, але не є навчальними запусками протоколу: моделі після 5 перевірочних кроків не оцінюються на validation, не зберігаються й не входять у таблицю пункту 3. Кожна перевірка починається з нового екземпляра моделі, побудованого за кроками 1–5 зі зерном 1.

Параметри. Для кожного з трьох режимів наведіть загальну кількість параметрів, кількість параметрів із `requires_grad=True` і склад груп оптимізатора: кількість параметрів у групі та її швидкість навчання. Кількість параметрів тут і далі - сума `numel()`, а не кількість тензорів. Числа мають збігтися з таблицею режимів. Переконайтеся, що жоден параметр із `requires_grad=False` не потрапив в оптимізатор.

Що змінюється під час навчання. Для режимів `frozen` і `unfreeze` (у `scratch` змінюється все, тож перевіряти там нічого):

1. Збережіть копію повного `state_dict()` - параметрів і буферів (`running_mean`, `running_var`, `num_batches_tracked`).
2. Викличте `set_train_mode` і виконайте 5 кроків оптимізатора на перших 5 пакетах навчального завантажувача.
3. Для кожного модуля верхнього рівня (`conv1`, `bn1`, `layer1`, …, `layer4`, `fc`) окремо перевірте, чи змінився хоч один його параметр і чи змінилася хоч одна ковзна статистика (running statistics) пакетної нормалізації в ньому. Порівняння - точне (`torch.equal`), без допуску.

Режим	Модуль	Параметри змінилися	Ковзні статистики BN змінилися	Збігається з очікуваним
		так / ні	так / ні / немає BN	так / ні

Очікувано: модулі зі стовпця «У `eval()`» таблиці режимів заморожені, тобто в них не змінилося **ні те, ні інше**; у решти модулів змінилися параметри, а в тих, що містять BN, - і ковзні статистики.

Дослід із навмисною помилкою. Для режиму `frozen` повторіть ту саму перевірку на новому екземплярі моделі, замінивши `set_train_mode` на звичайний `model.train()`. **Перед запуском** запишіть прогноз: що зміниться в замороженому кістяку, а що ні, і чи помітно це було б із кривої втрати на train. Зіставте результат із прогнозом. Наведіть максимальну абсолютну зміну `running_mean` і `running_var` у шарі `layer1.0.bn1` після 5 кроків. Поясніть, чому `requires_grad=False` не захищає ці значення і чому для `layer4` у режимі `unfreeze` їхнє оновлення правильне.

До основного експерименту переходьте після того, як усі перевірки збіглися з очікуваними.

---

3. Сім запусків

#	Режим	Зерно
1	`scratch`	1
2–4	`frozen`	1, 2, 3
5–7	`unfreeze`	1, 2, 3

У звіт і в розрахунки входять рівно ці сім запусків. Налагоджувальні запуски в таблицю не потрапляють і для вибору не використовуються.

Журнал запуску. Для кожного запуску збережіть `runs/<режим>_seed<зерно>/run.json` з:

- зерном і всіма налаштуваннями з розділів «Перетворення», «Модель і режими» та «Цикл навчання», включно з тими, що лишилися за замовчуванням;
- хешем коміту коду й позначкою, чи були незакомічені зміни;
- SHA-256 файлу `split.csv`, наприклад `hashlib.sha256(Path("split.csv").read_bytes()).hexdigest()`;
- версіями Python, PyTorch, torchvision, назвою пристрою;
- втратою на train і macro-F1 на validation за кожну епоху;
- часом навчання в секундах.

Хеш `split.csv` - запис версії даних: команда аналізу перевіряє, що він однаковий у всіх семи журналах і збігається з хешем поточного `split.csv`, і зупиняється з помилкою, якщо ні. Запуски на різних поділах порівнювати не можна; після будь-якої зміни поділу перенавчіть усі сім. Хеш коміту лише записується: різні коміти між запусками допустимі, але їх треба пояснити в README.

Поруч збережіть логіти validation після 8-ї епохи (`val_logits.npy`, масив 1498×6 у порядку рядків validation у `split.csv`). Ваги запусків зберігайте локально, доки не зроблено вибір пункту 5; до репозиторію додається лише вибраний. **На test у цьому пункті нічого не обчислюйте.**

Орієнтир для самоперевірки, а не критерій оцінювання: у режимі `frozen` macro-F1 на validation близько $0{,}88$. Значення нижче $0{,}80$ найчастіше означає помилку з BN або нормуванням. У `scratch` очікувано помітно менше; значення близько $1/6$ (вгадування) - ознака помилки, а не «поганої архітектури».

У README наведіть таблицю:

#	Режим	Зерно	Втрата train, епоха 8	macro-F1 validation, епоха 8	Час навчання, с

Побудуйте один рисунок із двох панелей - втрата на train і macro-F1 на validation за епохами - для всіх семи запусків на спільних осях.

---

4. Розклад ефектів

Для режимів `frozen` і `unfreeze` обчисліть середнє та розмах за трьома зернами:

$$\bar{F}_r = \frac{1}{3}\sum_{s=1}^{3} F_{r,s}, \qquad R_r = \max_s F_{r,s} - \min_s F_{r,s}.$$

Поріг шуму - більший із двох розмахів: $R = \max(R_{\text{frozen}},\ R_{\text{unfreeze}})$.

Ефект попереднього навчання та ефект розморожування:

$$\Delta_{\text{pre}} = \bar{F}_{\text{frozen}} - F_{\text{scratch},1}, \qquad \Delta_{\text{unf}} = \bar{F}_{\text{unfreeze}} - \bar{F}_{\text{frozen}}.$$

Їхня сума $\Delta_{\text{pre}} + \Delta_{\text{unf}} = \bar{F}_{\text{unfreeze}} - F_{\text{scratch},1}$ - повний виграш трансферу відносно навчання з нуля.

Величина	Значення
$\bar{F}_{\text{frozen}}$, R_{frozen}	
$\bar{F}_{\text{unfreeze}}$, R_{unfreeze}	
R	
Δ_{pre}	
Δ_{unf}	

Правило інтерпретації - окремо для Δ_{pre} і Δ_{unf}. Якщо $|\Delta| \le R$, різниця **не відрізняється від розкиду**: результату немає, і так це й треба записати. Якщо $|\Delta| > R$, ефект **вартий перевірки**, але не доведений.

Чому саме так. Нехай $F_{r,s}$ за різних зерен мають стандартне відхилення σ. Для трьох нормально розподілених значень $\mathbb{E}[R_r] \approx 1{,}69\,\sigma$, а стандартна похибка різниці двох середніх із трьох запусків $\sigma\sqrt{2/3} \approx 0{,}82\,\sigma$. Отже, правило $|\Delta| > R$ в середньому вимагає різниці понад дві стандартні похибки - воно консервативне. Водночас сам розмах трьох значень дуже неточний ($\mathrm{SD}(R_r) \approx 0{,}89\,\sigma$), тому для ефекту, близького до R, вердикт нестійкий. Це груба оцінка шуму, а не довірчий інтервал.

У звіті поясніть:

- що саме змінює зерно в кожному з трьох режимів і чому розкид у `frozen` і `unfreeze` може відрізнятися;
- чому R, виміряний на запусках 2–7, не характеризує розкид запуску 1 і чи змінює це висновок щодо Δ_{pre} (подивіться на криву запуску 1);
- чи однаковий бюджет режимів за часом навчання з вашої таблиці.

---

5. Вибір конфігурації і тестова оцінка

Вибір - лише за validation, у команді навчання. Конфігурація - режим із найбільшим $\bar{F}_r$ (для `scratch` - його єдине значення $F_{\text{scratch},1}$). Серед запусків цього режиму візьміть запуск із найбільшим $F_{r,s}$. Його ваги збережіть як `lab3_model.pt` - словник із ключами `state_dict`, `regime`, `seed`, `classes`, `img_size` (значення `img_size` - 160). Цей файл використовує ЛР4.

Якщо $|\Delta_{\text{unf}}| \le R$, вибір між `frozen` і `unfreeze` визначив шум. Процедура від цього не змінюється, але в звіті це треба назвати.

Test - один раз і лише в команді аналізу. Код, що завантажує test, викликається тільки з команди аналізу: вона перевіряє, що `lab3_model.pt` відповідає правилу вибору за збереженими журналами, і обчислює на CPU логіти test для цієї моделі. Повторний запуск команди аналізу для відтворення дозволений; результати test не використовуються для зміни режиму, запуску, гіперпараметрів чи кількості епох.

Наведіть macro-F1 на test і матрицю плутанини 6×6 (рядки - справжній клас, стовпці - передбачений, порядок класів фіксований) у вигляді таблиці й рисунка.

Порівняйте macro-F1 на test із $F_{r,s}$ вибраного запуску. Вибір найкращого з трьох запусків робить його валідаційне число оптимістичним. Чи видно це зміщення у ваших числах? Оцініть порядок шуму від скінченності вибірки: стандартна похибка частки правильних відповідей $\sqrt{p(1-p)/n}$ для $n = 1502$, де p - частка правильних відповідей (accuracy) вашої моделі на test. Поясніть, чому зміщення може бути непомітним або навіть мати протилежний знак.

П'ять характерних помилок. Візьміть п'ять найчастіших позадіагональних пар (справжній клас, передбачений клас) з матриці плутанини; за однакової кількості - у порядку зростання індексу справжнього, потім передбаченого класу. Для кожної пари візьміть файл test із такою помилкою, перший за іменем, відсортованим як рядок. Якщо різних пар менше п'яти, доберіть другий за іменем файл найчастішої пари, потім другої за частотою і так далі.

Файл	Справжній клас	Передбачений клас	Імовірність передбаченого класу	Кількість таких помилок у test

Імовірність - значення softmax логітів test для передбаченого класу. Збережіть рисунок із цими п'ятьма зображеннями. Для кожного одним реченням віднесіть помилку до однієї з категорій: сцена містить обидва класи; ймовірна помилка розмітки; помилка моделі на однозначному зображенні. Плутанину `glacier` ↔ `mountain` назвіть як властивість набору - «виправляти» її не потрібно.

---

6. Висновок

Завершіть звіт відповіддю на питання роботи окремо для Δ_{pre} і Δ_{unf}, у такому порядку: **напрям** ефекту, його **величина** в пунктах macro-F1, **співвідношення з розкидом** R. Кожне твердження підкріпіть числами з таблиць пунктів 3–5. Обидва ефекти виміряно за конкретного бюджету в 304 кроки - так і сформулюйте: з довшим навчанням `scratch` міг би скоротити відставання. Окремим реченням: який режим ви взяли б для нової задачі з 200 зображеннями на клас з урахуванням розкиду й часу навчання.

Що здати

Теку `lab3/` у власному GitHub-репозиторії всіх лабораторних робіт, яка містить:

- код у `.py`-файлах, файл залежностей із зафіксованими версіями та дві команди через `uv`, обидві з параметром шляху до локальної копії набору:
  - **навчання:** поділ, перевірки пунктів 1–2, сім запусків, журнали, вибір і збереження `lab3_model.pt`; test не використовується;
  - **аналіз:** перевірка хешу `split.csv` у журналах; зі збережених журналів - таблиця, криві, ефекти, перевірка вибору; потім тестова оцінка `lab3_model.pt` на CPU, матриця плутанини, п'ять помилок;
- `split.csv`;
- `runs/` з журналами й логітами validation усіх семи запусків;
- `lab3_model.pt` вибраного запуску (близько 45 МБ: у репозиторії в межах ліміту GitHub 100 МБ на файл або як файл GitHub Release з посиланням у README);
- рисунки: пакет після перетворень, криві семи запусків, матриця плутанини, п'ять помилок;
- `README` (md/pdf/docx) з результатами перевірок пунктів 1–2, прогнозом і результатом досліду з BN, таблицями пунктів 3–5, формулами ефектів і висновком.

Набір до репозиторію не додавайте. Відхилення від фіксованих налаштувань, якщо воно було вимушеним (наприклад, інша версія бібліотеки), назвіть у README; незадеклароване відхилення вважається помилкою реалізації.

Що саме має відтворюватися

На машині викладача запускаються обидві команди.

- **Аналіз** має дати ті самі числа, що у вашому README: таблицю семи запусків, $\bar{F}_r$, R_r, R, Δ_{pre}, Δ_{unf}, вибраний запуск, macro-F1 на test, матрицю плутанини й таблицю помилок. Тому всі тестові числа в README беріть із виводу команди аналізу.
- **Навчання** має відтворити той самий поділ (таблиця контролю), ту саму конфігурацію й ті самі відповіді «так / ні» в перевірках пункту 2; числові зміни статистик BN у досліді можуть відрізнятися в молодших розрядах. Значення macro-F1 можуть відрізнятися: обчислення на GPU і MPS недетерміновані й залежать від версій бібліотек. Абсолютного збігу не вимагається. Висновки, зроблені за вашими збереженими журналами, залишаються чинними.

Оцінювання та захист

Складова	Бали
Перевірки до навчання: поділ, пакет, L_0, режими навчання, дослід із BN	2
Сім запусків за протоколом, журнали, таблиця, криві й розклад ефектів	3
Вибір за validation, одноразова тестова оцінка, матриця плутанини й аналіз помилок	2
Висновок: напрям, величина ефектів і співвідношення з розкидом	2
Відтворюваність і повнота звіту	1
Разом	**10**

На захисті студент пояснює власний код, перевірки й числа. Потрібно вміти пояснити: чому заморожені модулі мають бути в режимі `eval()` і що відбувається інакше; навіщо кістяку менша швидкість навчання, ніж голові; що змінює зерно; чому різниця, менша за розкид, не є результатом; чому test обчислюється один раз.

Використання зовнішнього коду та генеративних інструментів дозволене із зазначенням джерел у `README`; воно не скасовує вимоги розуміти поданий розв'язок. Для повного зарахування результат має відтворюватися за наведеною командою на машині викладача.